



資料結構

Data Structure

Lab 6

姓名： 陳冠豪

學號： 114310125

Lab06-Q1

Blackjack (Use Queue)

Code

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <random>
#include <ctime>

using namespace std;

struct Player { // 玩家結構體，包含名稱和分數
    string name; // 玩家名稱
    int score; // 玩家分數
};

class Card { // 建立一個 Card 類別，用來表示單張撲克牌
public:
    string suit; // 花色
    string rank; // 點數
    Card() {}
    Card(string s, string r) : suit(s), rank(r) {}
    void display() const {
        cout << rank << " of " << suit << endl;
    }
};

class Deck { // 一整副牌（負責建立+洗牌）
private:
    Card cards[52]; // 52 張牌
    int index; // 目前發牌索引
public:
    Deck() : index(0) { // 初始化牌組
        string suits[] = { "Hearts", "Diamonds", "Clubs", "Spades" }; // 四種花色

        string ranks[] = { "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q",
            "K", "A" }; // 13 張牌
```

```

    int k = 0;
    for (int i = 0; i < 4; i++) { // 4 種花色
        for (int j = 0; j < 13; j++) { // 13 種點數
            cards[k] = Card(suits[i], ranks[j]);
            k++;
        }
    }
}

void shuffleDeck() { // 簡單洗牌 (隨機交換)
    for (int i = 51; i > 0; i--) { // 從最後一張牌開始，向前遍歷
        int j = rand() % (i + 1); // 生成隨機索引 j

        swap(cards[i], cards[j]); // 交換牌組中的兩張牌
    }
}

// 取得指定位置的牌
Card getCard(int i) const {
    return cards[i];
}
};

// Queue 類別
class Queue { // 用來當牌堆 (先進先出)
private:
    vector<Card> deck; // 用來存放卡牌的容器
    int Front;         // 指向牌堆的前端
    int Rear;          // 指向牌堆的尾端
    int capacity;      // 容器最大容量
public:
    Queue(int size) : Front(-1), Rear(-1), capacity(size) {}
    // 看有沒有滿
    bool isFull() const {
        return Rear >= capacity - 1;
    }
    // 看有沒有空
    bool isEmpty() const {
        return (Front == -1 && Rear == -1) || Front > Rear;
    }
    // 放一張牌到尾端

```

```

void enqueue(Card card) {
    if (isFull()) {
        cout << "牌堆已滿，無法插入！" << endl;
        return;
    }
    if (Front == -1) Front = 0;
    deck.push_back(card);
    Rear++;
}

//從前面抽掉一張牌
void dequeue() {
    if (isEmpty()) {
        cout << "牌堆為空，無法取出牌！" << endl;
        return;
    }
    Front++;

    //如果整個佇列用完就重置
    if (Front > Rear) {
        Front = -1;
        Rear = -1;
        deck.clear();
    }
}

//看最前面那張（不移除）
Card front() const {
    if (isEmpty()) {
        return Card("", "");
    }
    return deck[Front];
}

};

//建立+洗牌+塞進牌堆
void initializeDeck(Queue& cardDeck) {
    Deck deck;
    cout << "初始牌堆：\n";
    for (int i = 0; i < 52; i++) {
        deck.getCard(i).display();
    }
}

```

```

deck.shuffleDeck();

cout << "\n 洗牌後的牌堆: \n";
for (int i = 0; i < 52; i++) {
    deck.getCard(i).display();
}

//把洗好的牌全部丟進 queue
for (int i = 0; i < 52; i++) {
    cardDeck.enqueue(deck.getCard(i));
}
}

//發初始兩張牌
void initializePlayer(Player* player, string name, Queue& cardDeck) {
    player->name = name;
    player->score = 0;
    cout << player->name << " 抽到的牌: ";

    for (int i = 0; i < 2; i++) {
        Card drawnCard = cardDeck.front();
        cardDeck.dequeue();

        cout << drawnCard.rank << " of " << drawnCard.suit << " ";

        // 計算點數
        if (drawnCard.rank == "A") player->score += 1;
        else if (drawnCard.rank == "J" || drawnCard.rank == "Q" || drawnCard.rank
== "K") player->score += 10;
        else player->score += stoi(drawnCard.rank);
    }
    cout << "\n";
    cout << player->name << " 玩家目前分數: " << player->score << endl;
}

//玩家操作
void playerTurn(Player* player, Queue& cardDeck) {
    char choice;
    while (player->score < 21 && !cardDeck.isEmpty()) {
        cout << player->name << " 現在分數: " << player->score << " 要抽嗎? (h= 抽 ,
s = 不抽) ";
    }
}

```

```

    cin >> choice;

    if (choice == 'h') {
        Card newCard = cardDeck.front();
        cardDeck.dequeue();

        if (newCard.rank == "A") player->score += 1;
        else if (newCard.rank == "J" || newCard.rank == "Q" || newCard.rank ==
"K") player->score += 10;
        else player->score += stoi(newCard.rank);

        cout << player->name << " 抽到: " << newCard.rank << " of " <<
newCard.suit
            << " 總分: " << player->score << endl;

        if (player->score > 21) {
            cout << player->name << " 爆掉了!\n";
            return;
        }
    }
    else if (choice == 's') {
        cout << player->name << " 停牌，最終分數: " << player->score << endl;
        break;
    }
    else {
        cout << "輸入錯誤，請重新輸入\n";
    }
}
}

//莊家邏輯（小於17就抽）
void dealerTurn(Player* dealer, Queue& cardDeck) {
    while (dealer->score < 17 && !cardDeck.isEmpty()) {
        Card newCard = cardDeck.front();
        cardDeck.dequeue();

        if (newCard.rank == "A") dealer->score += 1;
        else if (newCard.rank == "J" || newCard.rank == "Q" || newCard.rank == "K")
dealer->score += 10;
        else dealer->score += stoi(newCard.rank);
    }
}

```

```

        cout << "莊家抽到：" << newCard.rank << " of " << newCard.suit
            << " 總分：" << dealer->score << endl;

        if (dealer->score > 21) {
            cout << "莊家爆了！玩家贏\n";
            return;
        }
    }
}
//比大小
void determineWinner(Player* player, Player* dealer) {
    if (player->score > 21) cout << player->name << " 爆了，莊家贏\n";
    else if (dealer->score > 21 || player->score > dealer->score)
        cout << player->name << " 贏了\n";
    else if (player->score == dealer->score) cout << "平手\n";
    else cout << "莊家贏\n";
}

int main() {
    srand(time(0));
    Queue cardDeck(52); //宣告牌堆
    initializeDeck(cardDeck); //初始化牌堆

    Player player, dealer; //宣告並初始化莊家以及玩家
    initializePlayer(&player, "玩家", cardDeck);
    initializePlayer(&dealer, "莊家", cardDeck);

    playerTurn(&player, cardDeck);
    if (player.score <= 21) {
        cout << "\n 莊家回合...\n";
        dealerTurn(&dealer, cardDeck);
        determineWinner(&player, &dealer);
    }

    return 0;
}

```

Result

初始牌堆：

2 of Hearts
3 of Hearts
4 of Hearts
5 of Hearts
6 of Hearts
7 of Hearts
8 of Hearts
9 of Hearts
10 of Hearts
J of Hearts
Q of Hearts
K of Hearts
A of Hearts
2 of Diamonds
3 of Diamonds
4 of Diamonds
5 of Diamonds
6 of Diamonds
7 of Diamonds
8 of Diamonds
9 of Diamonds
10 of Diamonds
J of Diamonds
Q of Diamonds
K of Diamonds
A of Diamonds
2 of Clubs
3 of Clubs
4 of Clubs

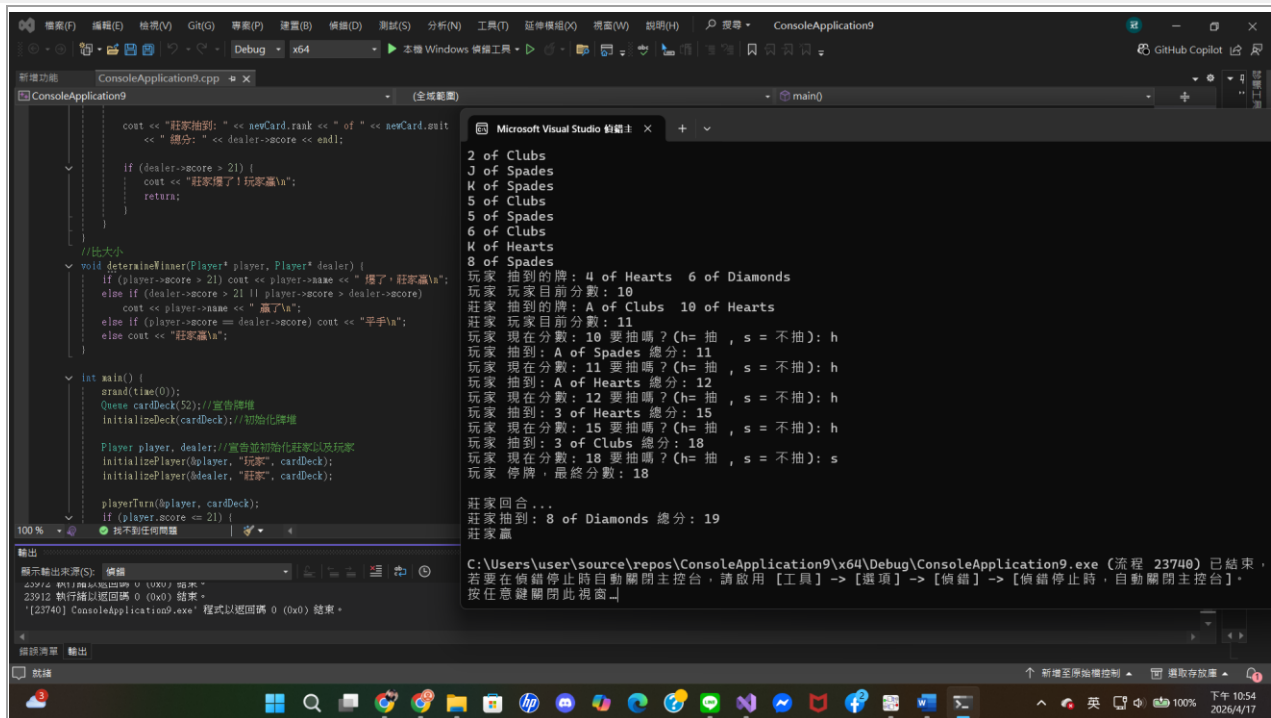
洗牌後的牌堆：

4 of Hearts
6 of Diamonds
A of Clubs
10 of Hearts
A of Spades
A of Hearts
3 of Hearts
3 of Clubs
8 of Diamonds
3 of Spades
K of Diamonds
8 of Clubs
2 of Hearts
Q of Clubs
J of Hearts
10 of Spades
2 of Spades
7 of Clubs
9 of Hearts
10 of Clubs
5 of Diamonds
4 of Clubs
2 of Diamonds
Q of Hearts
9 of Diamonds
Q of Spades
5 of Hearts
Q of Diamonds
J of Diamonds

2 of Clubs
J of Spades
K of Spades
5 of Clubs
5 of Spades
6 of Clubs
K of Hearts
8 of Spades
玩家抽到的牌：4 of Hearts 6 of Diamonds
玩家目前分數：10
莊家抽到的牌：A of Clubs 10 of Hearts
莊家目前分數：11
玩家現在分數：10 要抽嗎？(h= 抽 , s = 不抽)：h
玩家抽到：A of Spades 總分：11
玩家現在分數：11 要抽嗎？(h= 抽 , s = 不抽)：h
玩家抽到：A of Hearts 總分：12
玩家現在分數：12 要抽嗎？(h= 抽 , s = 不抽)：h
玩家抽到：3 of Hearts 總分：15
玩家現在分數：15 要抽嗎？(h= 抽 , s = 不抽)：h
玩家抽到：3 of Clubs 總分：18
玩家現在分數：18 要抽嗎？(h= 抽 , s = 不抽)：s
玩家停牌，最終分數：18

莊家回合...

莊家抽到：8 of Diamonds 總分：19
莊家贏



Discussion

在發牌的模擬中，Queue（佇列）和 Stack（堆疊）會因為資料存取方式不同產生不同的發牌結果。Queue 是先進先出（FIFO），也就是先放進去的牌會先被發出，因此可以維持洗牌後的原本順序，這種方式比較符合實際發牌的情況。而 Stack 是後進先出（LIFO），最後放進去的牌會最先被發出，會讓整個牌的順序顛倒，和一般發牌方式不太一樣。